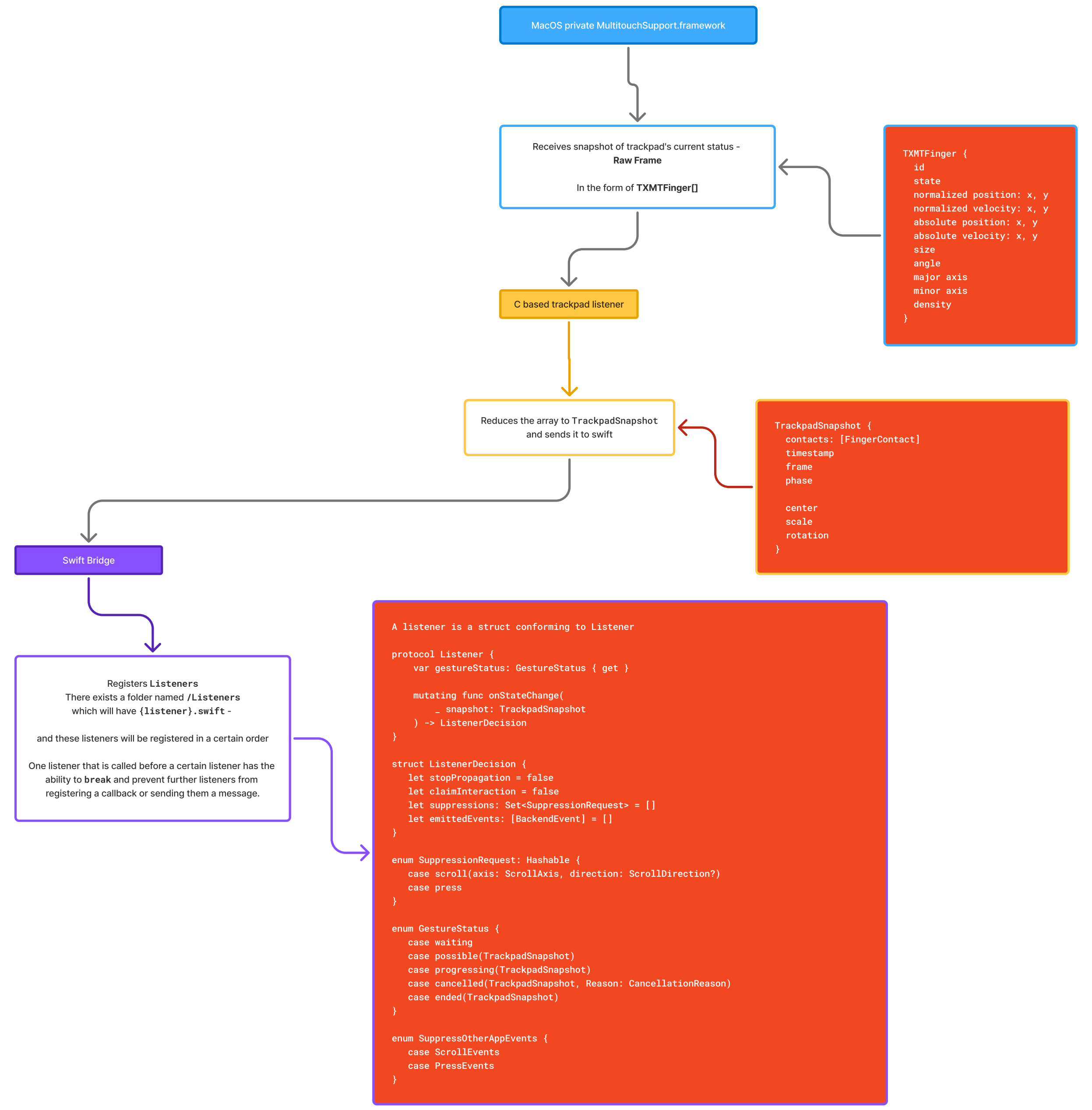


MacOS private MultitouchSupport.framework

Receives snapshot of trackpad's current status -
Raw Frame

In the form of **TXMTFinger[]**

```
TXMTFinger {
  id
  state
  normalized position: x, y
  normalized velocity: x, y
  absolute position: x, y
  absolute velocity: x, y
  size
  angle
  major axis
  minor axis
  density
}
```

C based trackpad listener

Reduces the array to `TrackpadSnapshot`
and sends it to swift

```
TrackpadSnapshot {
  contacts: [FingerContact]
  timestamp
  frame
  phase

  center
  scale
  rotation
}
```

Swift Bridge

Registers `Listeners`
There exists a folder named `/Listeners`
which will have `{listener}.swift` -

and these listeners will be registered in a certain order

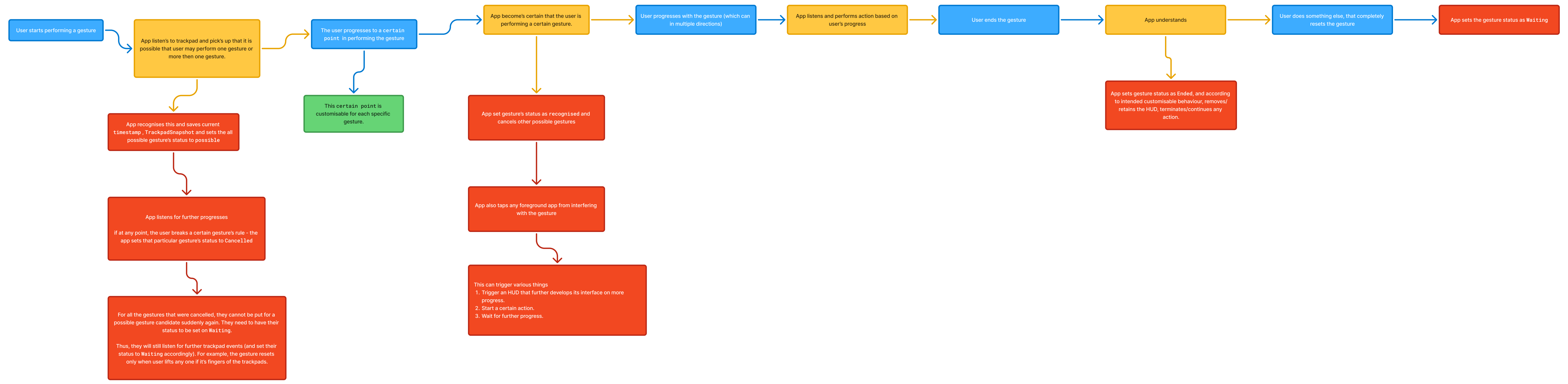
One listener that is called before a certain listener has the ability to `break` and prevent further listeners from registering a callback or sending them a message.

```
A listener is a struct conforming to Listener

protocol Listener {
    var gestureStatus: GestureStatus { get }

    mutating func onStateChange(
        _ snapshot: TrackpadSnapshot
    ) -> ListenerDecision
}

struct ListenerDecision {
   let stopPropagation = false
   let claimInteraction = false
   let suppressions: Set<SuppressionRequest> = []
   let emittedEvents: [BackendEvent] = []
}

enum SuppressionRequest: Hashable {
   case scroll(axis: ScrollAxis, direction: ScrollDirection?)
   case press
}

enum GestureStatus {
   case waiting
   case possible(TrackpadSnapshot)
   case progressing(TrackpadSnapshot)
   case cancelled(TrackpadSnapshot, Reason: CancellationReason)
   case ended(TrackpadSnapshot)
}

enum SuppressOtherAppEvents {
   case ScrollEvents
   case PressEvents
}
```

User experience for simple gestures

User's experience for simple gestures

Simple gesture means like -
1. Swiping from the edge/corners
2. Swiping along the edge
3. Simple linear gestures



User starts performing a gesture

App listen's to trackpad and pick's up that it is possible that user may perform one gesture or more then one gesture.

App recognises this and saves current `timestamp`, `TrackpadSnapshot` and sets the all possible gesture's status to `possible`

App listens for further progresses

if at any point, the user breaks a certain gesture's rule - the app sets that particular gesture's status to `Cancelled`

For all the gestures that were cancelled, they cannot be put for a possible gesture candidate suddenly again. They need to have their status to be set on `Waiting`.

Thus, they will still listen for further trackpad events (and set their status to `Waiting` accordingly). For example, the gesture resets only when user lifts any one if it's fingers of the trackpads.

The user progresses to a `certain point` in performing the gesture

This `certain point` is customisable for each specific gesture.

App become's certain that the user is performing a certain gesture.

App set gesture's status as `recognised` and cancels other possible gestures

App also taps any foreground app from interfering with the gesture

This can trigger various things
1. Trigger an HUD that further develops its interface on more progress.
2. Start a certain action.
3. Wait for further progress.

User progresses with the gesture (which can in multiple directions)

App listens and performs action based on user's progress

User ends the gesture

App understands

App sets gesture status as Ended, and according to intended customisable behaviour, removes/retains the HUD, terminates/continues any action.

User does something else, that completely resets the gesture

App sets the gesture status as `Waiting`

Frontend

Frontend HUDs

Performing certain gestures can trigger HUDs.

Any HUD will be an inherited class of `HUD` class

```
protocol HudDefinition {
   associatedtype Content: View

   var id: HUDID { get }
   var size: CGSize { get }

   func position(in context: HUDLayoutContext) -> CGPoint
   @ViewBuilder
   func content(context: HUDContext) -> Content
}
```

```
struct HUDLayoutContext {
  let mousePosition: CGPoint
  let screenFrame: CGRect
  let trackpadState: TrackpadState
}

struct HUDContext {
  let layout: HUDLayoutContext
  let state: HUDState
}
```

All the HUD will register by having a separate file `/HUDs/{hud_name}.swift`

Global state

Global states

will store
1. Currently active HUDs
2. Custom states if ever needed